

# Zadatak 2

## Igra Connect4

## Implementacija

Paralelna implementacija algoritma je ostvarena koristeći jedan glavni proces i proizvoljan broj radnika.

Radnici na početku svakog poteza traže zadatak i čekaju na odgovor. Taj odgovor može označavati kraj računanja trenutnog poteza, kraj igre ili može sadržavati zadatak. Ako je poruka zadatak onda ga odrađuju i šalju nazad rezultat, nakon čega čekaju novi zadatak. Kod 1 prikazuje programsko ostvarenje opisanog.

```
bool finish = false;
while (true) {
    if (finish) break;

    MPI_Send(nullptr, 0, MPI_BYTE, 0, TAG_SPECIAL, MPI_COMM_WORLD); // Ask for the first task
    while (true) {
        MPI_Probe(0, MPI_ANY_TAG, MPI_COMM_WORLD, &status); // Wait for a task
        if (status.MPI_TAG == TAG_SPECIAL) { // No more tasks
            MPI_Recv(nullptr, 0, MPI_BYTE, 0, TAG_SPECIAL, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            break;
        } else if (status.MPI_TAG == TAG_FINISH) { // The game is finished
            MPI_Recv(nullptr, 0, MPI_BYTE, 0, TAG_FINISH, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            finish = true;
            break;
        }
    }

    // Obtain task resources:
    MPI_Recv(spots.data(), config.width * config.height, MPI_CHAR, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
    int task_id = status.MPI_TAG;
    MPI_Recv(heights.data(), config.width * sizeof(std::size_t), MPI_BYTE, 0, MPI_ANY_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    board.set_board(spots, heights);

    double result = comp.move_recursive(board, config.solo_depth + 1); // Perform task
    MPI_Send(&result, 1, MPI_DOUBLE, 0, task_id, MPI_COMM_WORLD); // Send result
}
}
```

*Kod 1: Radna petlja radnika*

Glavni proces na početku poteza stvara dvije stablaste strukture kao što prikazuje Kod 2. Jedna struktura služi za pohranjivanje dobivenih rezultata i njihovo agregiranje prema roditeljskom čvoru kada se sakupe rezultati svih čvorova djece, a broj sakupljenih rezultata prati se drugom strukturom.

```
std::vector<std::vector<double>> buffer_results;  
std::vector<std::vector<std::size_t>> buffer_counts;  
std::size_t size = 1;  
for (std::size_t i = 0 ; i <= solo_depth + 1 ; i++) {  
    buffer_results.push_back(std::vector<double>(size, 0.0));  
    if (i != solo_depth + 1) buffer_counts.push_back(std::vector<std::size_t>(size, 0));  
    size *= width;  
}
```

*Kod 2: Stvaranje stablastih struktura za pohranu rezultata*

Glavni proces zatim rekurzivnom dubinskom pretragom pretražuje prostor stanja te kada dođe do zadane dubine za prosljeđivanje zadatka radnicima čeka da mu se radnik javi te od njega preuzima rješenje prethodno odrađenog zadatka i šalje novi zadatak što prikazuje Kod 3.

```
if (depth == solo_depth) {  
    MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);  
    if (status.MPI_TAG == TAG_SPECIAL) { // If asking for the first task  
        MPI_Recv(nullptr, 0, MPI_BYTE, status.MPI_SOURCE, TAG_SPECIAL, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
    } else { // We're getting a result  
        MPI_Recv(&result, 1, MPI_DOUBLE, status.MPI_SOURCE, status.MPI_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
    }  
  
    // Send new task:  
    MPI_Send(board.get_spots_data(), board.get_width() * board.get_height(), MPI_CHAR, status.MPI_SOURCE, task_idx_start + i, MPI_COMM_WORLD);  
    MPI_Send(board.get_heights_data(), board.get_width() * sizeof(std::size_t), MPI_BYTE, status.MPI_SOURCE, task_idx_start + i, MPI_COMM_WORLD);  
  
    if (status.MPI_TAG != TAG_SPECIAL) { // Store result  
        save_result_parallel(status.MPI_TAG, result, width, buffer_results, buffer_counts);  
    }  
} else {  
    move_recursive_parallel(board, (task_idx_start + i) * width, buffer_results, buffer_counts, depth + 1);  
}
```

*Kod 3: Komunikacija glavnog procesa u rekurzivnoj funkciji*

Ispis zadnjih poteza igre u kojoj računalu pobjeđuje prikazan je na Slika 1.

```

. . X . . . .
. . 0 0 . . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X . 0
0 1 2 3 4 5 6
Your move: 5
. . X . . . .
. . 0 0 . . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X X 0
0 1 2 3 4 5 6
. . X . . . .
. . 0 0 0 . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X X 0
0 1 2 3 4 5 6
Your move: 1
. . X . . . .
. X 0 0 0 . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X X 0
0 1 2 3 4 5 6
. . X . 0 . .
. X 0 0 0 . .
. X 0 0 0 . .
. 0 0 X X . X
. X X 0 X . 0
. X 0 X X X 0
0 1 2 3 4 5 6
Computer wins!

```

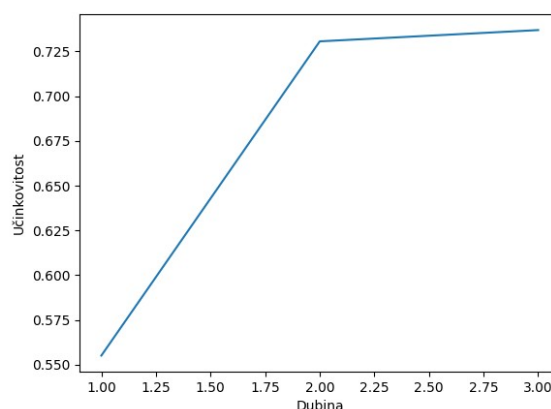
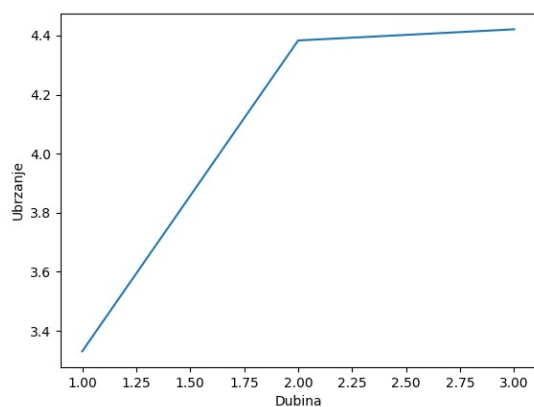
*Slika 1: Ispis  
zadnjih nekoliko  
poteza igre u  
kojoj pobjeđuje  
računalo*

## Kvantitativna analiza

Konstante mjerenja:

- širina ploče = 7
- visina ploče = 6
- dubina pretraživanja = 10
- broj procesa = 6 (1 glavni + 5 radnika)

| Dubina - # zadataka | Trajanje (ms) | Ubrzanje | Učinkovitost |
|---------------------|---------------|----------|--------------|
| 0 - 1 (seq)         | 69209         | -        | -            |
| 1 - 7               | 20780         | 3,3306   | 0,5551       |
| 2 - 49              | 15789         | 4,3834   | 0,7306       |
| 3 - 343             | 15654         | 4,4212   | 0,7369       |



Ubrzanje i učinkovitost u ovome slučaju imaju značajan skok kada se dubina aglomeracije poveća s 1 na 2, a zatim manji skok kada se poveća na 3. Ovo smanjenje dobitaka je očekivano jer sitnozrnatost zadataka znači sve manje, a količina komunikacije se značajno povećava te se uz to povećava i količina posla kojeg izvodi glavni proces.